

Static Test Review Pack — Money Transfer

UC-TR-01 · Transfer Money to Another Account · Static Testing (Verification Only)

Version	1.0	Review date	13-Sep-2026
Reviewer	Alaa Shehata	Classification	Review — Rework Required
Feature	Transfer Money to Another Account (UC-TR-01)	Status	REJECTED — REWORK REQUIRED

Static verification only is stated on the cover, scope, and summary. No code was executed; all findings are static observations. The pack identifies UC-TR-01 and the money-transfer feature, traces the use case, alternative flows, and post-conditions, and records exactly two confirmed defects.

Executive Result

STATUS: REJECTED — REWORK REQUIRED

Checklist: **9 Pass / 5 Fail** | Confirmed defects: **2** | Severity: **1 Critical / 1 Major**

Testing performed: **Static verification only** — no code execution. Scope: source/destination validation, amount validation, balance logic, debit/credit, transaction recording, update calls, return messages, failure-state safety.

Severity legend: **Critical** | **Major** | **Pass** | Informational

Review Baseline — UC-TR-01

Review objective (verbatim): Verify, without executing the code, that the `BankingService.transfer` implementation conforms to the approved UC-TR-01 use case, its alternative flows, balance-safety rules, transaction behavior, and failure post-conditions.

Scope — in scope (static verification only): source-account validation, destination-account validation, amount validation, balance logic, debit/credit behavior, transaction recording, update calls, return messages, and failure-state safety. **Explicitly excluded:** runtime, integration, performance, security, and database verification because the assignment requires static testing only.

Entry: Approved scenario and use case are available; Java implementation is submitted; review checklist is prepared.

Exit: Every checklist item has Pass/Fail/Not Applicable evidence; all confirmed defects have reports; the summary includes counts and a rework decision; the pack is reviewed for consistency and exported successfully.

Baseline summary: money transfer between two accounts in the same bank (Authenticated Customer). Main steps 1–10 (amount > 0; retrieve both accounts; ACTIVE checks; balance ≥ amount; debit; credit; record transaction; success message). Alternative flows A1 (Invalid amount), A2/A3 (not ACTIVE), A4 (Insufficient funds), A5 (Destination account not found). Post-conditions: success changes balances + records transaction; failure changes no balances.

Assumptions vs. observed facts: the Java source as supplied contains no line numbers, so locations are cited by method/statement. Static review indicates exactly two confirmed defects (DEF-TR-01 Critical/High; DEF-TR-02 Major/High). The source account is a stated precondition ("Source account exists"); a source-account null guard is at most an additional robustness observation, never counted as a third confirmed defect.

Requirements-to-Code Traceability Matrix

Severity legend: **Critical** | **Major** | **Pass** | Informational

Baseline item	Expected behavior	Code evidence	Result	Finding
Main step 1 / A1	Amount must be greater than zero	if (amount <= 0)	Pass	—
Main step 3	Retrieve both accounts	Two fetchAccount(...) calls	Pass	—
Main step 4 / A2	Source account must be ACTIVE	"ACTIVE".equals(fromAccount.getStatus())	Pass*	Null robustness reviewed separately
Main step 5 / A3	Destination account must be ACTIVE	"ACTIVE".equals(toAccount.getStatus())	Pass*	Depends on destination object being present
Main step 6 / A4	Reject when source balance is less than amount	fromAccount.getBalance() > amount	Fail	DEF-TR-01
Main step 7	Debit source by amount	setBalance(balance - amount)	Pass	Safe only after balance defect is fixed
Main step 8	Credit destination by amount	setBalance(balance + amount)	Pass	—
Main step 9	Record transfer transaction	saveTransaction(...)	Pass	—
Main step 10	Return success message	return "Transfer successful"	Pass	—
A5 / failure post-condition	Missing destination must return a clear message and change no balances	No toAccount == null guard before getStatus()	Fail	DEF-TR-02

Note on Pass*: used only when a row is structurally present but depends on the missing-object risk covered by DEF-TR-02. The implementation contains the expected flow structure, but static verification fails because the insufficient-funds decision is inverted and the destination-not-found alternative flow is not safely implemented.

Completed Static Checklist (14 items)

Severity legend: **Critical** | **Major** | **Pass** | Informational

#	Check item	Result	Evidence / remarks
1	Amount > 0 validation is present	Pass	amount <= 0 returns Invalid amount
2	Invalid amount alternative flow is mapped	Pass	Covers A1
3	Both source and destination accounts are retrieved	Pass	Two fetch calls are present
4	Source account ACTIVE check is present	Pass	Uses constant-first string comparison
5	Destination account ACTIVE check is present	Pass*	Check exists; null-path safety fails separately
6	Destination-not-found flow is handled	Fail	Missing null guard; DEF-TR-02
7	Insufficient balance condition is correct	Fail	Operator is inverted; DEF-TR-01
8	Source debit operation is present	Pass	Source balance is reduced
9	Destination credit operation is present	Pass	Destination balance is increased
10	Transfer transaction is recorded	Pass	saveTransaction is called
11	Both account updates are requested	Pass	Two updateAccount calls are present

#	Check item	Result	Evidence / remarks
12	Success message is clear	Pass	Returns Transfer successful
13	Failure leaves balances unchanged	Fail	The inverted balance rule can reach mutation with insufficient funds; destination null path is unsafe
14	Code follows defensive/static quality expectations	Fail	Missing null guard; logic defect; source null behavior noted as a risk if precondition is violated

Metric	Value
Total checklist items	14
Pass	9 (item 5 Pass* counted as Pass)
Fail	5
Not Applicable	0

Counting rule: 9 Pass / 5 Fail / 0 Not Applicable treats item 5 (Pass*) as Pass because the ACTIVE check is structurally present; its null-path safety failure is counted separately under items 6, 13, and 14. Only two confirmed defects are counted.

Checklist result: FAIL. The method is structurally complete but is not statically verified because the insufficient-funds rule is inverted and the destination-not-found flow is unhandled.

Inspection / Review Report

Method: checklist-based inspection plus requirements traceability. **Participants:** Senior Test Engineer / Reviewer (leads inspection, records findings); Peer Developer (checks observations against structure); Junior Developer / Author (context, rework).

Severity legend: **Critical** | **Major** | **Pass** | Informational

Defect ID	Severity	Priority	Location	Requirement / flow	Description	Recommendation	Status
DEF-TR-01	Critical	High	Sufficient-balance check in BankingService.transfer()	Main success step 6, Alternative Flow A4, and the failure post-condition	Inverted insufficient-balance condition: the code checks <code>fromAccount.getBalance() > amount</code> , so valid transfers can be rejected while transfers exceeding the source balance can continue to debit and credit accounts	Change the relational operator to <code><</code> and re-review the surrounding debit path	Open
DEF-TR-02	Major	High	After <code>fetchAccount(toAccountId)</code> , before <code>toAccount.getStatus()</code>	Alternative Flow A5 and the failure post-condition	Missing destination-account null handling: the code dereferences <code>toAccount</code> without a null guard, so the destination-not-found flow is not implemented safely	Add a null check immediately after fetching the destination account, before any status access, returning "Destination account not found" with no balance changes	Open

Notes (static observations only): checklist counts 9 Pass / 5 Fail, 0 Not Applicable (item 5 counted as Pass). Confirmed defects: 2 (1 Critical / 1 Major). Source-account null handling is an additional robustness observation only, never counted as a third confirmed defect.

Overall result: REJECTED — REWORK REQUIRED. The implementation must be corrected and submitted for a focused static re-review. No runtime pass/fail claim is made because execution is explicitly out of scope.

Individual Defect Reports

Severity legend: **Critical** | **Major** | **Pass** | Informational

DEF-TR-01 — Inverted insufficient-balance condition

Title	Inverted insufficient-balance condition
Type	Functional logic defect
Severity	Critical
Priority	High
Detected by	Static inspection
Location	Sufficient-balance check in <code>BankingService.transfer()</code>
Expected	Reject only when <code>fromAccount.getBalance() < amount</code> .
Actual	The code checks <code>fromAccount.getBalance() > amount</code> .
Impact	Valid transfers can be rejected, while transfers that exceed the source balance can continue to debit and credit accounts.
Requirement / flow violated	Main success step 6, Alternative Flow A4, and the failure post-condition.
Recommendation	Change the relational operator to <code><</code> and re-review the surrounding debit path.
Status	Open

Developer acceptance criteria:

- Code correction: replace the `>` operator with `<` in the sufficient-balance check so insufficient funds are rejected.
- Focused static re-review: re-verify the corrected condition and the surrounding debit path against main success step 6, Alternative Flow A4, and the failure post-condition, without executing the code.
- Confirmation that the correction does not change the successful transfer sequence or failure-state guarantee (on success: debit source, credit destination, record transaction, return success message; on failure: no balance changes).

DEF-TR-02 — Missing destination-account null handling	
Title	Missing destination-account null handling
Type	Robustness / missing alternative-flow handling
Severity	Major
Priority	High
Detected by	Static inspection
Location	After <code>fetchAccount(toAccountId)</code> , before <code>toAccount.getStatus()</code>
Expected	Return "Destination account not found" when the destination account does not exist, with no balance changes.
Actual	The code dereferences <code>toAccount</code> without a null guard.
Impact	The destination-not-found flow is not implemented safely; a missing account can cause a <code>NullPointerException</code> before the required message is returned.
Requirement / flow violated	Alternative Flow A5 and the failure post-condition.
Recommendation	Add a null check immediately after fetching the destination account, before any status access.
Status	Open

Developer acceptance criteria:

- Code correction: add a destination-account null guard immediately after `fetchAccount(toAccountId)`, before any `toAccount.getStatus()` access, returning "Destination account not found" with no balance changes.
- Focused static re-review: re-verify the added guard and the destination validation path against Alternative Flow A5 and the failure post-condition, without executing the code.
- Confirmation that the correction does not change the successful transfer sequence or failure-state guarantee (on success: debit source, credit destination, record transaction, return success message; on failure: no balance changes).

Note: the source account is a stated use-case precondition ("Source account exists"); a source-account null guard is at most an additional robustness observation for defensive programming and is never counted as a third confirmed defect in this pack.

Static Test Summary Report

STATUS: REJECTED — REWORK REQUIRED

Checklist: **9 Pass / 5 Fail** | Confirmed defects: **2** | Severity: **1 Critical / 1 Major**
 Testing performed: **Static verification only**

Metric	Value
Total checklist items	14
Pass	9
Fail	5
Not Applicable	0
Confirmed defects	2
Critical	1 (DEF-TR-01)
Major	1 (DEF-TR-02)
Review status	REJECTED — REWORK REQUIRED

Risk and recommendation: static review indicates that the Critical defect (DEF-TR-01) threatens account-balance integrity: valid transfers can be rejected, while transfers exceeding the source balance can continue to debit and credit accounts, violating the failure post-condition. Static review further indicates that the Major defect (DEF-TR-02) prevents the required destination-not-found response. Recommendation: correct both defects (change the balance

operator to <; add the destination-account null guard returning "Destination account not found"), then conduct a focused static re-review. Proceed to dynamic test design and execution only as a later phase outside this assignment.

Limitations: no runtime, integration, persistence, concurrency, performance, or security conclusions can be made from this static review alone. The Java code was not executed; all findings are static observations, and no runtime pass/fail claim is made.

Appendix — Annotated Code Excerpt and Review Notes (Static Observations)

The excerpt below is a static observation aid only. Annotations [A] and [B] mark the two confirmed defects; they are review notes, not execution evidence.

```
public String transfer(String fromAccountId, String toAccountId, double amount) {
    if (amount <= 0) return "Invalid amount";                // A1 OK
    Account fromAccount = fetchAccount(fromAccountId);
    Account toAccount = fetchAccount(toAccountId);
                                // [B] missing guard here (see below)
    if (!"ACTIVE".equals(fromAccount.getStatus()))
        return "Source account not active";                // A2 OK
    if (!"ACTIVE".equals(toAccount.getStatus()))
        return "Destination account not active";            // A3 OK
    // [A] >>> if (fromAccount.getBalance() > amount)        // DEF-TR-01
    // [A] >>>     return "Insufficient funds";
    fromAccount.setBalance(fromAccount.getBalance() - amount); // debit
    toAccount.setBalance(toAccount.getBalance() + amount);    // credit
    saveTransaction(fromAccountId, toAccountId, amount);
    updateAccount(fromAccount);
    updateAccount(toAccount);
    return "Transfer successful";
}
```

Annotation	Static observation
[A] DEF-TR-01	[A] DEF-TR-01: `>` is inverted – must be `<`
[B] DEF-TR-02	[B] DEF-TR-02: missing `if (toAccount == null) return "Destination account not found";` before getStatus()

Review notes: (1) Source-account precondition note — the use case states the source account exists, so a source-account null guard is an additional robustness observation for defensive programming only, never counted as a third confirmed defect. (2) Pass* counting rule — checklist counts 9 Pass / 5 Fail / 0 Not Applicable treat item 5 (Pass*) as Pass because the destination ACTIVE check is structurally present; its null-path safety failure is counted separately under items 6, 13, and 14. Only DEF-TR-01 (Critical/High) and DEF-TR-02 (Major/High) are confirmed defects. Status: REJECTED — REWORK REQUIRED.